Practicum: apparaat communicatie met Python

Leerdoelen

- ☐ Kan communiceren met basis labapparatuur via Python
- ☐ Kan uitleggen welke stappen nodig zijn voor communicatie met labapparatuur.
- ☐ Is bekend en kan overweg met de PyVisa-package van Python.
- ☐ De student kan een overzichtelijke map- en modulestuctuur gebruiken en opzetten voor instrumentcommunicatie, passend bij een groter Python-project.

Veiligheid

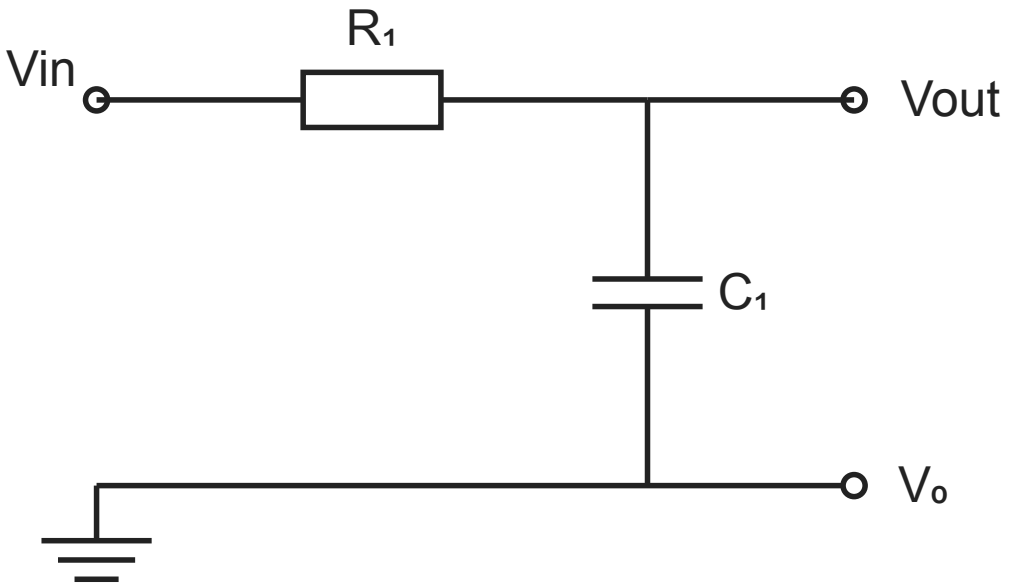
1 We werken deze practicumreeks met elektrische schakelingen. Zorg ervoor dat je deze goed aansluit voordat je verder gaat met experimenteren. Anders kan er kortsluiting ontstaan in de apparatuur. Laat je werk checken vooraf.

Historische ontwikkeling van apparaatuuraansturing

Het betrouwbaar uitvoeren en registreren van metingen vormt het hart van elk experiment. De manier waarop meetinstrumenten worden aangestuurd, heeft de afgelopen decennia een sterke ontwikkeling doorgemaakt. Aanvankelijk gebeurde dit volledig handmatig: instellingen werden aangepast met draaiknoppen en schakelaars, en resultaten werden afgelezen op analoge meters of oscilloscopen. De eerste automatiseringsstappen kwamen met seriële communicatieprotocollen zoals **RS-232** en later **GPIB** (General Purpose Interface Bus), waarmee instrumenten centraal vanaf een computer konden worden aangestuurd. Programmeertalen als *Fortran*, *C* en later *LabVIEW* werden gebruikt voor het verzenden van commando's en het uitlezen van data, meestal via relatief laag-niveau communicatie die gebruikers handmatig moesten programmeren.
Met de komst van microcontrollers veranderde de interne aansturing van apparaten ingrijpend. Deze compacte chips combineren processor, geheugen en I/O en voeren specifieke taken uit, zoals het regelen van spanningen of uitlezen van sensoren. Veel meetinstrumenten kregen interne microcontrollers, terwijl aan de buitenzijde seriële of GPIB-communicatie gehandhaafd bleef. Tegelijkertijd maakten goedkope ontwikkelplatforms zoals de **Arduino** en **Raspberry-Pi** het mogelijk om zelf eenvoudig meet- en regelsystemen te bouwen, wat vooral in onderwijs en kleinschalig onderzoek veel werd toegepast.
Tegenwoordig is apparaatuuraansturing grotendeels gestandaardiseerd via de **VISA**-standaard (Virtual Instrument Software Architecture), die zowel klassieke verbindingen (RS-232, GPIB) als moderne interfaces zoals **USB** en **Ethernet** ondersteunt. Hierdoor kan met één softwarelaag een breed scala aan instrumenten worden aangestuurd. Een belangrijke recente ontwikkeling is de opkomst van Python als universeel hulpmiddel. Met bibliotheken als `PyVISA` kunnen instrumenten direct via GPIB, USB of netwerk worden bestuurd. Waar vroeger dure propriëtaire software nodig was, volstaat nu vaak een korte Python-script en een standaardinterface. In deze practicumserie gaan we aan de slag met apparaat-communicatie via Python. De schakeling die wij gaan aansturen betreft een passief RC-filtersysteem.

Passieve RC-filtersystemen

Een **passief RC-filter** is een eenvoudig elektrisch netwerk van weerstanden en condensatoren waarmee de frequentierespons van een signaal kan worden aangepast, zonder actieve componenten zoals versterkers. Zulke filters worden veel toegepast om ongewenste frequenties te onderdrukken of juist door te laten, bijvoorbeeld om ruis te verwijderen of een meetsignaal te conditioneren voordat het naar een meetinstrument gaat. In figuur 1 staat een voorbeeld elektrische schakeling van een laagdoorlaatfilter.



Figuur 1: Laagdoorlaatfilter: R in serie, C naar massa; de uitgangsspanning wordt over C gemeten.

Het gedrag van een RC-filter kan worden beschreven met behulp van de impedantie van een condensator,

$$Z_C(\omega) = \frac{1}{i\omega C},$$

waarbij $\omega = 2\pi f$ de hoekfrequentie is en i de imaginaire eenheid. Voor een eenvoudig **laagdoorlaatfilter**, bestaande uit een weerstand R in serie en een condensator C naar massa, volgt voor de overdrachtsfunctie

$$H(\omega) = \frac{V_{\text{uit}}}{V_{\text{in}}} = \frac{1}{1 + i\omega RC}.$$

Bij lage frequenties ($\omega \ll 1/RC$) is $\mid H \mid \approx 1$ en wordt het signaal vrijwel ongehinderd doorgelaten. Bij hoge frequenties ($\omega \gg 1/RC$) neemt de amplitude af als $1/\omega$, zodat snelle variaties en ruis worden onderdrukt. De overgangsfrequentie, of **afsnijfrequentie**, wordt gegeven door

$$f_c = \frac{1}{2\pi RC}.$$

Om van de overdrachtsfunctie naar een **Bode-plot** te gaan, splits je $H(\omega)$ op in een amplitude- en fasegedeelte. De amplitude wordt uitgedrukt in decibel,

$$\mid H(\omega) \mid_{\text{dB}} = 20 \log_{10} \mid H(\omega) \mid,$$

en de fase als

$$\phi(\omega) = \arg\big(H(\omega)\big).$$

Voor het laagdoorlaatfilter hierboven volgt eenvoudig

$$\mid H(\omega) \mid = \frac{1}{\sqrt{1 + (\omega RC)^2}}, \qquad \phi(\omega) = -\arctan(\omega RC).$$

Door deze twee functies uit te zetten tegen de logaritme van de frequentie ontstaat de Bode-plot: een amplitude-grafiek die de verzwakking per frequentie laat zien en een fase-grafiek die aangeeft

hoe sterk het signaal wordt vertraagd. Helaas kunnen wij de fase-verschuiving niet meten met een **Digitale Multi Meter (DMM)**, hiervoor zouden wij een oscilloscoop moeten gebruiken. Deze grafische weergave maakt het filtergedrag bij lage, overgangs- en hoge frequenties direct zichtbaar. Naast **laagdoorlaatfilters** bestaan er ook **hoogdoorlaatfilters** door de positie van weerstand en condensator om te wisselen. In dat geval worden juist lage frequenties onderdrukt en hoge frequenties doorgelaten. Met combinaties van meerdere RC-netwerken kunnen **banddoorlaat- of bandstopfilters** worden gerealiseerd.

Laagdoorlaat en hoogdoorlaat onderscheiden

Meet de uitgangsspanning steeds tussen het knooppunt van R en C en massa.

- **Laagdoorlaat:** R in serie met de ingang, C naar massa. Meet de uitgang over C.
- **Hoogdoorlaat:** C in serie met de ingang, R naar massa. Meet de uitgang over R.

Voor het hoogdoorlaatfilter geldt

$$H_{\text{hoog}}(\omega) = \frac{i\omega RC}{1 + i\omega RC}.$$

Lage frequenties worden onderdrukt; hoge frequenties worden doorgelaten. Beide filters hebben een afsnijfrequentie $f_c = 1/(2\pi RC)$, waar de amplitudeverhouding $1/\sqrt{2}$ is (ongeveer -3 dB). Deze formules veronderstellen een ideale spanningsbron en een uitgang die niet merkbaar wordt belast door het meetinstrument.

Voorbereidingsopdracht 1

- Installeer **NI-Max** op jouw laptop Link[1].
- Maak een nieuwe virtuele omgeving in Anaconda en installeer hierop de volgende packages: "pyvisa", "pyvisa-py","numpy", "matplotlib", "ipykernel" & "Pyserial?", gebruik Python versie 3.11.
- Download (of clone) de github repository voor de aansturing van de apparaten via Link.
- Maak een schematische tekening van een zelfgekozen passief filtersysteem.
- Schrijf een meetplan voor doormeten van een jouw filter (bode-plot).

[1] Je zult hiervoor een account moeten aanmaken. Dit is helaas de harde realiteit dat niet alles écht open-source is.

Practicum

Opdracht 1(a)

In deze eerste opdracht gaan we de het filter bouwen en deze handmatig doormeten. Voordat we gaan automatiseren moeten we immers zeker weten dat onze apparatuur goed aangesloten staat.

1. Bouw op het breadboard jouw gekozen filter en sluit de functiegenerator en digitale multimeter aan. Controleer bij jouw practicumbegeleider of deze goed aangesloten staan.
2. Meet, met de hand, bij enkele verschillende frequenties het in-en uitgangssignaal en bereken de overdrachtsfunctie $\mid H(\omega) \mid$ op deze frequenties. Gebruik hiervoor een (RMS) voltage van $V_{rms} = 2$ volt.
3. Schrijf jouw resultaten op in je digitale labjournaal.

Opdracht 1(b)

Nu we zeker weten dat het filtersysteem werkt gaan we deze aansturen via Python.

1. Sluit de digitale multimeter en de functiegenerator aan via de USB-B kabel.
2. Open NI-MAX en zoek de bijbehorende apparaat ID'en van de apparatuur.
3. Maak een nieuwe Jupyter Notebook met de naam `rc_filter.ipynb` en plaats deze op de juiste locatie binnen jouw projectstructuur (zie Figuur 2).[2]
4. Controleer nu met onderstaande code of je kan communiceren met jouw apparatuur. **Let op!** De functiegenerator (fg) en de digitale multimeter (dmm) kunnen omgewisseld zijn.

Met deze code controleer je of je kan communiceren met je meetapparatuur. Daarnaast zorgt deze code ervoor dat de communicatiepoorten geopend zijn. Dit is een belangrijk details, want, na gebruik moeten wij ook de apparatuur weer sluiten.

Test communicatie meetapparatuur

```
import pyvisa
rm = pyvisa.ResourceManager()

fg = rm.open_resource(rm.list_resources()[0])
dmm = rm.open_resource(rm.list_resources()[1])

#Controleer of de functiegenerator (fg) en de digitale multimeter (dmm) niet
omgewisseld zijn.
print("Instrument gevonden:", fg.query("*IDN?"))
print("Instrument gevonden:", dmm.query("*IDN?"))
```

Nu we zeker weten dat we kunnen communiceren met onze labapparatuur, is het belangrijk om de code duidelijk en onderhoudbaar te organiseren. Het doel is om niet telkens handmatig ieder instrument aan te sturen, maar gebruik te maken van herbruikbare functies en modules die dit voor ons afhandelen.
Een overzichtelijke manier om dit te structureren is met een *boomstructuur*, waarbij elke map een duidelijke rol binnen het project heeft. Bovenin bevinden zich de algemene modules en documentatie; dieper in de structuur staan de instrument-drivers, hulpfuncties en mappen voor data-opslag. Deze indeling maakt het eenvoudiger om de code te begrijpen, te testen en later uit te breiden. In figuur 2 is te zien hoe deze onderdelen logisch zijn gegroepeerd. De map `instruments` bevat bijvoorbeeld de drivers voor verschillende meetinstrumenten, terwijl `utils` de ondersteunende functies bundelt die in meerdere modules worden gebruikt. De map `data` is gereserveerd voor ruwe meetdata en

[2] In professionele software-engineering wordt doorgaans niet met notebooks gewerkt, maar uitsluitend met `.py`-modules. Voor dit practicum maken we een uitzondering, omdat we zowel de meetcode als de gegenereerde data en visualisaties in één bestand willen kunnen bekijken.

gegenereerde plots. Deze indeling maakt het eenvoudiger om onderdelen te vinden, te testen en uit te breiden, en zorgt ervoor dat de communicatiecode voor verschillende instrumenten consistent blijft opgebouwd.

```
Communicatie/
├── instruments/
│   ├── __init__.py
│   ├── hardware.py
│   ├── rigol_dg1022.py
│   └── rigol_dm3058e.py
├── utils/
│   ├── __init__.py
│   └── functions.py
├── data/
│   ├── raw/
│   └── plots/
├── rc_filter.py(nb)
├── README.md
└── .gitignore
```

Figuur 2: Boomdiagram voor apparaatcommunicatie.

Opdracht 1(c)

Nu we zeker weten dat we kunnen communiceren met onze labapparatuur via Python, kunnen we de eerste testmetingen gaan uitvoeren.

1. Stel **via Python** een voltage in van 2 Volt (Root Mean Squared) bij de functiegenerator met een frequentie van 100 Hz. Controleer op de display van de functiegenerator of deze correct staat ingesteld. Gebruik hiervoor de onderstaande code. Zie je niets gebeuren? Dan kan het zijn dat jouw de functiegenerator en digitale multimeter omgewisseld zijn!
2. Leg uit wat iedere regel doet en schrijf dit op in je labjournaal.

Functiegenerator aansturen met Python

```
from instruments.hardware import open_instruments
from instruments.rigol_dg1022 import set_fixed_sine

rm, fg, dmm = open_instruments(message = 1)

#Aansturing functigenerator
f = 100 #Hz
vrms_output = 2.0 #Volt
set_fixed_sine(fg, freq=f, vrms=vrms_output)
```

Opdracht 1(c)

De volgende stap is het doen van een meting via Python. Hiervoor gebruiken wij de digitale multimeter.

1. Meet het ingangsvoltage en het uitgangsvoltage 5 keer. Gebruik hiervoor als inspiratie onderstaande code.
2. Sla de data op in de dictionary `results`.

Digitale multimeter aansturen met Python

```
from instruments.rigol_dm3058e import measure_vrms, measure_vrms_n
from utils.functions import save_as_csv

#Start meting digitale multimeter
vals = measure_vrms_n(dmm,5)

#opslaan in een nette structuur
results = {}
results[f] = {
            "samples": vals,
            "mean": float(vals.mean()),
            "std": float(vals.std(ddof=1)),
            "n": len(vals)
        }
save_as_csv(results, "data/raw/input_voltage.csv")
```

Opdracht 1(c)

De laatste stap nu de data weer inladen.

1. Laad de data in. Gebruik hiervoor als inspiratie onderstaande code.
2. Schrijf de gemeten gemiddelde waarde en standaard fout in je labjournaal.

Uitlezen data

```
from utils.functions import load_from_csv
data = load_from_csv("data/raw/input_voltage.csv")

freqs_in  = list(data.keys())
V_mean    = [v["mean"]    for v in data.values()]
V_std     = [v["std"]     for v in data.values()]
samples   = [v["samples"] for v in data.values()]
```

Opdracht 1(c)

1. Sluit de apparatuur nu weer af met Python. Zie onderstaande code.

Afsluiten meetapparatuur

```
from instruments.hardware import close_instruments
close_instruments(rm, fg, dmm, message=1)
```

Opdracht 1(d)

1. Maak een bode-plot van jouw gekozen filtersysteem.
2. Bepaal de afsnijfrequentie aan de hand van jouw **bode-plot** en vergelijk deze met jouw gekozen waarde. Je hoeft hiervan alleen te kijken naar de amplitude en dus niet naar de fase[3].

[3] Dat kan ook niet in dit geval, daarvoor hebben we een oscilloscoop nodig.

Afronding

1. Zet de gevonden figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2. Vul de **Resultaat- en Discussie**-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van jouw experiment (met onzekerheid) staan en een korte discussie of deze waarde **realistisch** en **betrouwbaar** is.

Appendix 1: Drivers & virtuele omgeving

Voor communicatie tussen Python en laboratoriumapparatuur (zoals oscilloscopen en functiegeneratoren) wordt gebruikgemaakt van de VISA-standaard (*Virtual Instrument Software Architecture*). Deze standaard definieert een uniforme interface voor het aanspreken van meetinstrumenten via verschillende fysieke verbindingen, zoals USB, GPIB en LAN.

VISA, PyVISA en drivers

- **VISA-driver (bijv. NI-VISA, Keysight VISA, TekVISA, R&S VISA):** Dit is de onderliggende software die zorgt voor de fysieke communicatie met het instrument. Zie dit als de *postdienst* die weet hoe een bericht (via USB, LAN, GPIB) op de juiste plek komt.
- **PyVISA (Python package):** Dit is een Python-interface naar de VISA-driver. PyVISA stuurt geen data rechtstreeks naar de hardware, maar gebruikt de geïnstalleerde VISA-library (bijv. `visa64.dll`) als tussenlaag.
- **SCPI (Standard Commands for Programmable Instruments):** Dit is de *taal* waarin de instrumenten commando's ontvangen, zoals `MEAS:VOLT?` of `FREQ 1KHZ`. Vrijwel alle moderne meetinstrumenten ondersteunen SCPI.
- **NI-MAX:** NI Measurement & Automation Explorer is een hulpmiddel om apparaten te detecteren en resource strings (zoals `USB0::0x1AB1::::::INSTR`) te achterhalen. PyVISA kan ook zonder NI-MAX werken, mits het juiste adres van het instrument bekend is.

Naast NI-VISA kunnen ook andere VISA-implementaties worden gebruikt (Keysight, Tektronix, Rohde & Schwarz, of Rigol UltraSigma).

NI-MAX

NI Measurement & Automation Explorer (NI-MAX) is een hulpprogramma van National Instruments waarmee je:

- Apparaten kunt detecteren en configureren die via GPIB, USB, LAN of andere bussen verbonden zijn.
- Resource strings kunt achterhalen die nodig zijn voor communicatie via `PyVISA`.
- Instrumenten kunt testen door eenvoudige SCPI-commando's te sturen.

NI-MAX wordt meestal geïnstalleerd samen met NI-VISA, de driver die `PyVISA` gebruikt om daadwerkelijk met het instrument te communiceren.

Tip: Na installatie kun je in NI-MAX je Rigol oscilloscoop vinden en de juiste USB-resource string kopiëren voor gebruik in Python.

Installatie en virtuele omgeving

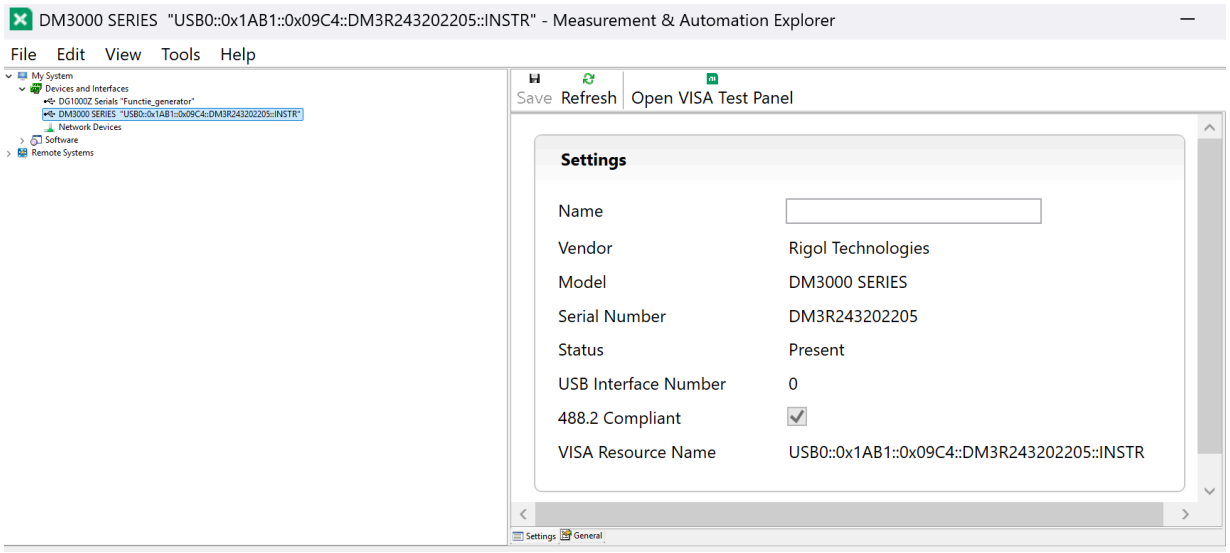
Het is aan te raden om te werken in een virtuele Python-omgeving en de benodigde packages via `pip` of `conda-forge` te installeren:

Anaconda Prompt: virtuele omgeving en installatie

```
# Maak een nieuwe virtuele omgeving via Anaconda Prompt of Navigator
(base) C:\Users\student> conda create -n Practicum2_3 python=3.11
(base) C:\Users\student> conda activate Practicum2_3

# Installeer PyVISA en optioneel de pure Python backend
(Practicum2_3) C:\Users\student> pip install pyvisa pyvisa-py numpy
               matplotlib scipy ipykernel
```

Appendix 2: Materialen en tips



Figuur 3: Startscherm van NI-Max software. Apparaat ID start met "USB".

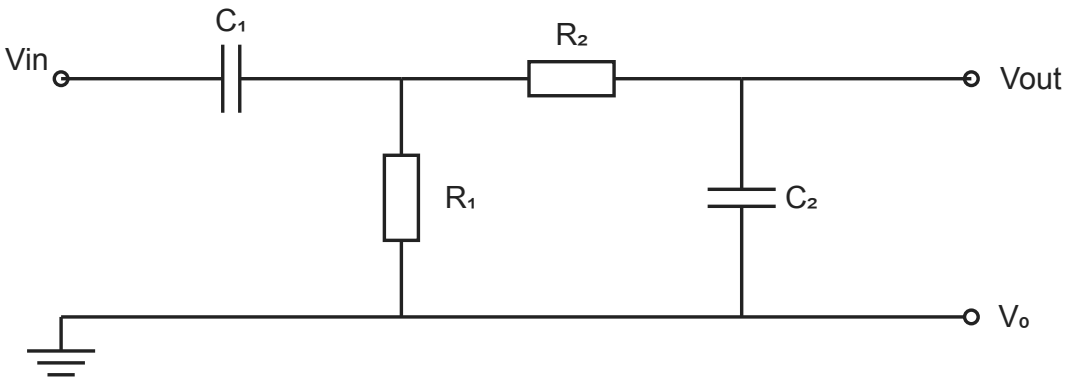
Tabel 1: Veelgebruikte SCPI-commando's voor de Rigol DG1022 / DG1022Z functiegenerator.

Doel	SCPI-commando
Selecteer kanaal	`SOUR1`, `SOUR2`
Golfvorm kiezen	`SOUR1:FUNC SIN`
Amplitude in Vpp	`SOUR1:VOLT <value>`
Amplitude in Vrms	`SOUR1:VOLT:UNIT VRMS`
Offset instellen	`SOUR1:VOLT:OFFS <value>`
Frequentie instellen	`SOUR1:FREQ <value>`
Fase instellen	`SOUR1:PHAS <deg>`
Output aan	`OUTP1 ON`
Output uit	`OUTP1 OFF`
Output laden instellen	`OUTP1:LOAD <ohm>`
Bestaande instellingen uitlezen	`SOUR1:VOLT?`, `SOUR1:FREQ?`
Burst-mode aan/uit	`SOUR1:BURS:STAT ON`
Modulatie AM/FM/PWM	`SOUR1:AM:STAT ON`, `SOUR1:FM:STAT ON`
Noise waveform	`SOUR1:FUNC NOIS`
Square-wave duty cycle	`SOUR1:PULS:DCYC <percent>`
Ramp symmetry	`SOUR1:FUNC:RAMP:SYMM <0–100>`
Systeem reset	`*RST`
Identificatie opvragen	`*IDN?`

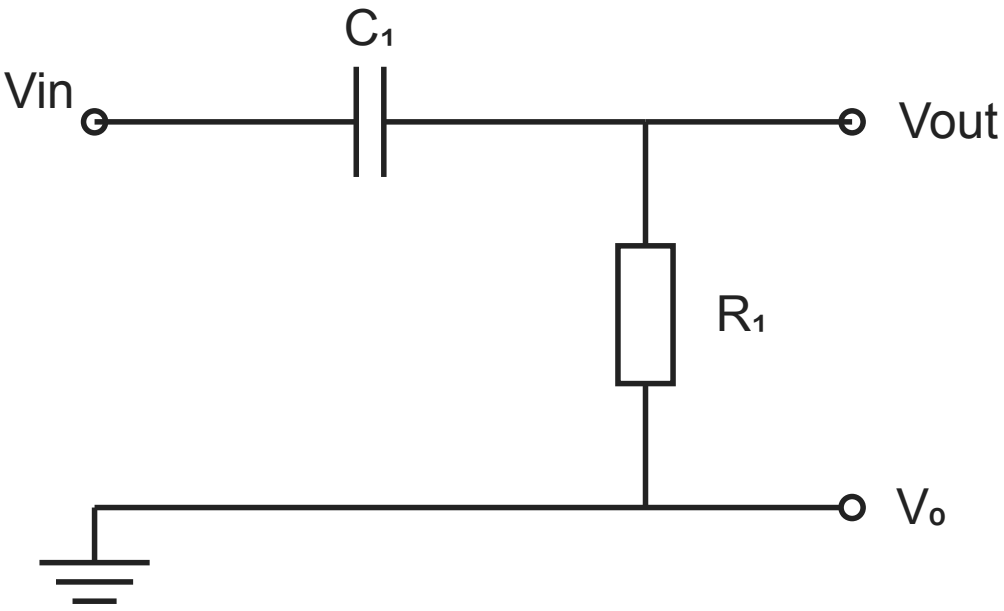
Tabel 2: Veelgebruikte SCPI-commando's voor de Rigol DM3058E digitale multimeter.

Doel	SCPI-commando
Identificatie opvragen	`*IDN?`
Reset instrument	`*RST`
Foutenbuffer leegmaken	`*CLS`
Stop lopende metingen	`ABORt`
AC-spanning meten (Vrms)	`CONF:VOLT:AC`
DC-spanning meten	`CONF:VOLT:DC`
Wisselspanning uitlezen (ad-hoc)	`MEAS:VOLT:AC?`
Gespacificeerde meting uitlezen	`READ?`
Frequentie meten	`CONF:FREQ`
Weerstand meten	`CONF:RES`
Stroom meten (AC/DC)	`CONF:CURR:AC`, `CONF:CURR:DC`
Triggerbron instellen	`TRIG:SOUR IMM`
Aantal metingen per trigger	`TRIG:COUN <n>`
Aantal samples instellen	`SAMP:COUN <n>`
Acquisitie starten	`INIT`
Buffer uitlezen	`FETCh?`
NPLC instellen (integratietijd)	`VOLT:AC:NPLC <x>`
Automatische bereikkeuze uit	`VOLT:AC:RANG:AUTO OFF`
Bereik instellen	`VOLT:AC:RANG <range>`
Auto-zero uit	`ZERO:AUTO OFF`
Detectieband AC	`VOLT:AC:DET:BAND 20` / `100`
Display aan/uit	`DISP:ENAB ON`, `DISP:ENAB OFF`
Status opvragen	`STAT:OPER:COND?`
Fouten opvragen	`SYST:ERR?`

Appendix 3: Schematische tekeningen schakelingen



Figuur 4: Schema van het banddoorlaatfilter (bandpass).



Figuur 5: Hoogdoorlaatfilter: C in serie, R naar massa; de uitgangsspanning wordt over R gemeten.

Appendix 4: apparaataansturing

Voorbeeld: functiegenerator aansturen met Python

```python
def set_fixed_sine(gen, freq=100, vrms=5.0, channel=1):
    """
    Stel de functiegenerator in op een vaste sinusgolf.
    Inclusief expliciet aan- en uitzetten van de uitgang.
    """

    chan = f"SOUR{channel}"

    gen.write(f"{chan}:FUNC SIN")           # sinusgolf kiezen
    gen.write(f"{chan}:VOLT:UNIT VRMS")     # amplitude in Vrms
    gen.write(f"{chan}:VOLT {vrms}")        # waarde instellen
    gen.write(f"{chan}:FREQ {freq}")        # frequentie instellen

    gen.write(f"OUTP{channel} ON")          # uitgang inschakelen

    # Optioneel: gegevens teruggeven, maar NIET verplicht:
    # return {"freq": freq, "vrms": vrms, "channel": channel}

# ------------------------------------------------------------
# Voorbeeldgebruik:
# ------------------------------------------------------------
import pyvisa

rm = pyvisa.ResourceManager()
gen = rm.open_resource("USB0::0x1AB1::0x0642::DG1ZAxxxxxxx::INSTR")

set_fixed_sine(gen, freq=100, vrms=5.0, channel=1)

gen.close()
```